SOFTWARE ENGINEERING

Chapter 9: Design Concepts

Based on: Software Engineering: A Practitioner's Approach (9th Edition)

Authors: Roger S. Pressman and Bruce R. Maxim

Student Name: Asal Ghafouri
Instructor: Engineer Rasoulzadeh
Course: Software Engineering

1. Introduction

Software design is the process that transforms requirements into a representation of the software that can be implemented. It is the place where customer needs, business constraints, and technical considerations come together.

Chapter 9 of Pressman and Maxim's book presents the fundamental design concepts that every software engineer should understand before applying specific design methods. These concepts form the foundation for creating high-quality, maintainable systems.

This chapter explains the nature of design, important quality guidelines, and core design concepts such as abstraction, modularity, architecture, patterns, information hiding, and functional independence. Examples continue with the online shopping application.

2. The Nature of Software Design

Software design encompasses the set of principles, concepts, and practices that lead to the development of a high-quality system. Design principles establish an overriding philosophy, while design concepts provide the foundation on which methods and practices are built.

Design is not optional. The alternative to good design is bad design, not the absence of design. A well-considered design reduces complexity, improves quality, and makes later changes easier and less risky.

3. Design Quality Guidelines

A good design should:

- Exhibit an architecture that has been created using recognizable architectural styles or patterns
- Be modular — the software should be logically partitioned into elements that perform specific functions
- Contain distinct representations of data, architecture, interfaces, and components
- Lead to data structures that are appropriate for the classes to be implemented
- Lead to components that exhibit independent functional characteristics
- Lead to interfaces that reduce the complexity of connections between components and with the external environment
- Be derived using a repeatable method driven by information obtained during requirements analysis

- Be represented in a way that communicates its meaning effectively

4. Fundamental Design Concepts

A set of fundamental concepts has evolved over decades of software engineering practice. Understanding these concepts is essential before applying detailed design techniques.

4.1 Abstraction

Abstraction allows a designer to focus on essential characteristics while suppressing unnecessary detail. Different levels of abstraction exist — from a high-level statement of the problem to detailed procedural or data descriptions.

Example: At a high level, the shopping system is described as “an application that allows customers to browse products and place orders.” At lower levels, the designer specifies the structure of a ShoppingCart class and the algorithms for calculating totals and applying discounts.

4.2 Architecture

Software architecture refers to the overall structure of the software, the ways in which components interact, and the structure of data used by the components. A good architecture provides conceptual integrity for the system.

4.3 Patterns

A design pattern describes a recurring design problem and a proven solution for it. Patterns capture expertise and make it reusable. They describe the problem, the context, the solution structure, and the consequences of applying the pattern.

4.4 Modularity

Modularity is the division of software into separately named and addressable components (modules) that are integrated to satisfy requirements. Modular design makes systems easier to understand, test, and maintain.

4.5 Information Hiding

Information hiding means that modules are designed so that information (data and implementation details) contained within a module is inaccessible to other modules that have no need for that information. This reduces the impact of changes.

4.6 Functional Independence

Functional independence is achieved by designing modules with high cohesion (a module performs a single, related set of tasks) and low coupling (modules have limited and well-defined interactions). Independent modules are easier to develop, test, and reuse.

4.7 Refinement

Refinement is a process of elaboration. A program is developed by successively refining levels of procedural detail. Starting from a high-level statement, the designer gradually adds more detail until an implementation is reached.

4.8 Refactoring

Refactoring is the process of changing a software system in such a way that it does not alter the external behavior of the code, yet improves its internal structure. The goal is to make the design cleaner, more readable, and easier to maintain without adding new functionality.

4.9 Design Classes

As design progresses, analysis classes are refined into design classes that contain more implementation-oriented detail. Design classes often include attributes with data types, operations with signatures, and relationships that reflect the chosen architecture.

5. The Design Model

The design model is the representation of the software that will be built. It typically includes:

- Data / Class design – transforms analysis classes into implementation classes and data structures
- Architectural design – defines the relationships among the major structural elements of the software
- Interface design – describes how software elements, hardware elements, and end-users communicate
- Component-level design – transforms structural elements into procedural descriptions of software components

These four design activities are interrelated and are often performed iteratively.

6. Practical Example: Design Concepts in the Online Shopping Application

Applying the design concepts to the shopping application:

6.1 Abstraction and Refinement

The team begins with a high-level description of the system and gradually refines it. Early design focuses on major subsystems (catalog, cart, payment, order management). Later iterations add detailed class designs and algorithms.

6.2 Architecture and Modularity

The architecture is organized into layers or modules such as Presentation, Application Services, Domain, and Data Access. Each module has a clear responsibility, supporting modularity and information hiding.

6.3 Patterns

The team applies well-known patterns: a Repository pattern for data access, a Strategy pattern for different payment methods, and an Observer-like approach for notifying the user of order status changes.

6.4 Information Hiding and Independence

The payment processing details are hidden inside a PaymentService module. Other parts of the system interact with it only through a narrow interface. This keeps coupling low and makes it easier to replace or extend payment providers later.

6.5 Refactoring

After the first working version of the cart is complete, the developers refactor the code to remove duplicated calculation logic and to improve naming, without changing the external behavior of the cart.

7. Conclusion

Chapter 9 presents the fundamental concepts that underlie all good software design. Abstraction, architecture, patterns, modularity, information hiding, functional independence, refinement, and refactoring together provide a powerful intellectual toolkit.

When these concepts are understood and applied, design becomes more than a mechanical translation of requirements. It becomes a creative yet disciplined activity that produces systems that are easier to build, understand, test, and maintain.

In the online shopping application, applying these concepts leads to a cleaner architecture, more independent modules, and a design that can evolve as new features are added.

8. References

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.